

Refinzi Internal Architecture Record

Version: v0.1 (Frozen Foundation)

Status: APPROVED

Date: June 2026

Core Company Philosophy

Refinzi is not a prompt library. Refinzi is not a model router. Refinzi is not a writing assistant.

Refinzi is an **Ambient AI Interaction Layer**.

Its job is simple:

Help humans express intent to any LLM better than they naturally can.

The user should feel:

"I simply wrote what I meant. Refinzi helped the AI understand me."

The user should never think about:

- Prompt engineering
- XML
- Tags
- Templates
- AI optimization
- Compiler stages

All complexity stays invisible.

Product Positioning

External

Your thoughts. Your AI. Better understood.

Alternative:

The intelligent layer between you and AI.

PMF Thesis

Bad inputs create bad outputs.

Refinzi silently upgrades user intent before it reaches the model.

No learning curve. No prompt engineering courses. No templates required.

Things We Will NOT Build Before PMF

- ✗ Context Graph
 - ✗ Multi-provider routing engine
 - ✗ Agent framework
 - ✗ Enterprise features
 - ✗ Settings-heavy UI
 - ✗ Thread management
 - ✗ Transform logs
 - ✗ Capability routing
 - ✗ Metadata registry
 - ✗ Conversation memory
-

Current User Experience

User selects text.

✨ Orb appears.

Single Click: → Polish / Add Context

Long Hold: → Default Expert Prompt

No menus.

No popup complexity.

No prompt selection.

No cognitive load.

Expert Prompt Philosophy

Hold ✨

Automatically injects the default expert prompt.

If users later want custom prompts:

Type:

```
/1  
/2  
/3
```

inside their selection.

Those IDs resolve from Prompt Library.

No UI required.

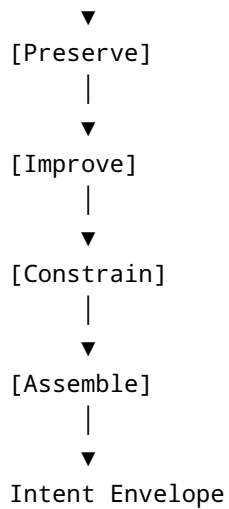
History and Prompt Library are linked.

No duplicate Save system.

Output Intelligence Engine

Compiler Pipeline

```
User Input  
|
```



Stage Responsibilities

Preserve

Extract:

- Original meaning
- Original intent
- Original language
- Original personality
- Original writing style

Never modify source.

Improve

Add:

- Missing context
- Better structure
- Better vocabulary
- Useful constraints
- Better output expectations

Never add facts.

Constrain

Apply:

- No hallucination
 - Preserve personality
 - Preserve language
 - Preserve objective
 - Minimal modification
-

Assemble

Build provider-independent Intent Envelope.

No XML.

No GPT formatting.

No Claude formatting.

No Gemini formatting.

Compiler Philosophy

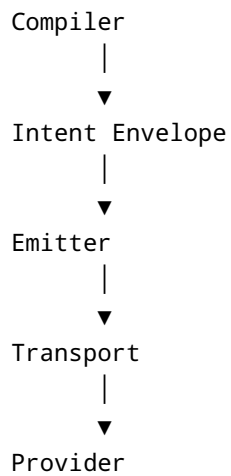
Compiler owns:

- Human intent
- Meaning preservation
- Structural optimization

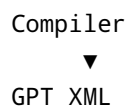
Adapters (future) own:

- Provider formatting
 - Wire protocols
 - XML
 - Markdown
 - JSON
 - Transport
-

Future Architecture



NOT:



Envelope Contract (Frozen v1)

```
interface RefinziEnvelopeV1 {  
  
    readonly schemaVersion: 1;  
  
    readonly compilerVersion: 1;  
  
    readonly envelopeId: string;  
  
    readonly createdAt: number;  
  
    readonly rawIntent: string;  
  
    readonly locale: string;  
  
    intentAnnotations: {  
  
        detectedRole?: string;  
  
    }  
}
```

```
        detectedOutput?: string;

        confidence: number;

    };

    inputConstraints: {};

    outputConstraints: {};

    outputPolicy: {

        preserveMeaning: true;

        preserveLanguage: true;

        preservePersonality: true;

        minimalModification: true;

    };

    extensions: {};

}
```

Envelope Rules

Immutable

- rawIntent
- envelopeId
- createdAt
- compilerVersion
- schemaVersion

Never mutated.

No Metadata Field

Rejected.

Use:

```
extensions {}
```

only.

Constraints Split

inputConstraints

Examples:

- language boundary
- token budget
- domain scope

outputConstraints

Examples:

- output length
 - JSON required
 - markdown required
-

Versioning Rules

schemaVersion: Major structural changes.

compilerVersion: Compiler improvements.

Migration framework reserved.

No migration engine implemented yet.

XML Decision

Users never see XML.

Compiler never generates XML.

Future emitters may serialize:

GPT: XML

Claude: Markdown

Gemini: JSON

Local: Plain text

Envelope remains unchanged.

Privacy Strategy

Day 1:

- Zero backend
- Zero user database
- Zero prompt storage
- Zero document storage
- Zero analytics server

Refinzi is local software.

BYOK.

Provider APIs communicate directly.

Company servers are optional and currently nonexistent.

Future Learning Engine

NOT Context Graph.

Day 1 learning:

- Prompt history
- Expert prompt reuse
- User accepted prompts

All local.

No cloud.

Future Moat

Everything below is commodity:

- Prompt libraries
- Context expansion
- Multi-provider support
- BYOK
- XML prompting

Potential moat:

Local-first Personal Context Graph.

But:

NO implementation before PMF.

Founder Operating Rules

Rule 1

Ship before architecting.

Rule 2

Every feature must satisfy at least one:

- Improves output quality
- Improves debugging
- Enables future local learning

Otherwise reject.

Rule 3

Never spend 2 hours debugging assumptions.

Audit first.

Git history first.

Repository evidence first.

Only then code.

Rule 4

One provider abstraction.

Many providers later.

Do not optimize for hypothetical users.

Rule 5

Invisible sophistication.

The smarter Refinzi becomes, the simpler it should feel.

Cline / Gemini Operating Rules

Before coding, always provide:

1. Files to create
2. Files to modify
3. Public API
4. Regression risk
5. Assumptions
6. Smallest possible implementation

No coding until approval.

Company North Star

Refinzi should eventually become:

The operating system between human thought and artificial intelligence.

But Version 1 only needs to prove one thing:

A better structured thought produces a noticeably better AI output.

Everything else can wait.